

Relatório de Estudo de Caso 01: Problema do Caixeiro Viajante

Gabriel Santos Faria
Guilherme Augusto Pires Lopes
João Henrique Alves Martins
João Victor Otoni Gonçalves
Thales Faria Almeida
Thiago Henrique Silva de Almeida

April 24, 2026

1 Aplicações Práticas do TSP

Para as aplicações, seguem 2 aplicações de TSP clássico, 2 mais atuais com restrições específicas, e uma modelagem aplicada ao contexto da UFMG.

1.1 Aplicação de um Viajante (Problema Clássico)

Essa aplicação é a mais clássica e famosa de TSP, e aborda o problema base dessa modelagem. Antes da popularização dos algoritmos computacionais modernos, o planejamento logístico de rotas comerciais de longa distância dependia de aproximações ineficientes. O marco que transformou essa realidade ocorreu quando os pesquisadores George Dantzig, Ray Fulkerson e Selmer Johnson, através do artigo *Solution of a Large-Scale Traveling-Salesman Problem* (1954) [Dantzig et al. \(1954\)](#), buscaram a rota rodoviária mais curta que conectasse Washington D.C. e as capitais dos outros 48 estados (contíguos) dos Estados Unidos.

Na formalização matemática que estabeleceu o padrão do problema, o mapa rodoviário americano foi modelado como um grafo não direcionado $G = (V, E)$. O conjunto de vértices V corresponde às 49 cidades, enquanto o peso das arestas E representa a distância rodoviária c_{ij} entre a cidade i e a cidade j . O objetivo do modelo era o clássico do problema original: percorrer todas as cidades com o menor custo possível. Para resolver essa instância, que na época possuía um número astronômico de rotas possíveis ($\frac{48!}{2}$), os autores desenvolveram um método chamado planos de corte (*cutting-plane method*), resolvendo o problema à mão e comprovando matematicamente que uma rota de 12.815 milhas era a mínima.

1.2 Aplicação na Indústria

Em qualquer indústria, a eficiência na linha de montagem é fundamental para otimizar o custo de produção. Um dos gargalos desse processo ocorre na fabricação de placas metálicas, mais especificamente “Placas de Circuito Impresso” (PCBs), onde uma máquina de controle numérico (CNC) precisa realizar milhares de furos milimétricos para a inserção de componentes.

Como o tempo de ciclo é dominado pelo movimento mecânico da broca entre um furo e outro, otimizar essa trajetória é um grande desafio. Para resolver esse problema, pesquisadores como Magirou (1986), no artigo *The Efficient Drilling of Printed Circuit Boards* [Magirou \(1986\)](#), aplicaram o TSP. Na modelagem desse problema, a placa é representada como um plano cartesiano que forma um grafo $G = (V, E)$. O conjunto de vértices V corresponde às coordenadas exatas de cada furo que

deve ser feito, enquanto o peso das arestas E representa a distância de deslocamento entre dois pontos i e j . O objetivo do modelo é calcular o ciclo hamiltoniano ótimo, encontrando a rota mais curta que permita à máquina realizar todos os furos e retornar à posição inicial, minimizando o tempo de produção de cada placa.

1.3 Aplicação Genética

No estudo genético, o sequenciamento de genomas é um desafio algorítmico, pois os equipamentos de laboratório não conseguem ler uma fita de DNA inteira de uma só vez. Em vez disso, o material genético é quebrado em milhares de fragmentos curtos, que são lidos de maneira desordenada.

O desafio de remontar a sequência original a partir desses pedaços sobrepostos é um problema massivo de otimização combinatória. Para solucionar essa questão, o pesquisador Richard Karp (1996) [Karp \(1996\)](#) modelou a montagem do genoma como uma variação do TSP clássico, porém de forma assimétrica, ou seja, c_{ij} pode ser diferente de c_{ji} .

Nessa modelagem, os fragmentos de DNA formam um grafo direcionado $G = (V, E)$. O conjunto de vértices V representa cada fragmento individual. O peso c_{ij} das arestas direcionadas E quantifica o grau de incompatibilidade (falta de sobreposição) entre o final do fragmento i e o início do fragmento j . Quanto maior essa sobreposição entre dois fragmentos, menor o “custo” da aresta que os conecta. O objetivo do algoritmo é calcular o caminho hamiltoniano (não retorna ao nó de origem) ótimo para reconstruir a cadeia original do DNA, minimizando o custo total de incompatibilidade e gerando a supercadeia mais curta possível.

1.4 Aplicação no Esporte

Em campeonatos esportivos, a tarefa de produzir a tabela de jogos é de extrema importância, pois uma configuração equilibrada é essencial para garantir a isonomia da competição. Como o volume de viagens e as distâncias percorridas afetam diretamente a fadiga e a recuperação física dos atletas, formular uma tabela justa torna-se um desafio crítico.

Para resolver esse problema, os pesquisadores Easton, Nemhauser e Trick, através do artigo *The Traveling Tournament Problem: Description and Benchmarks* (2001) [Easton et al. \(2001\)](#), modelaram o cenário como uma adaptação do clássico TSP. Devido à inclusão de restrições rígidas de calendário, a formulação foi denominada *Traveling Tournament Problem* (TTP).

Na formalização matemática, o campeonato é representado por um grafo $G = (V, E)$, onde os nós V (vértices) representam as cidades-sede dos times, e as arestas E possuem um peso associado c_{ij} , que representa a distância real de viagem entre a cidade i e a cidade j . O objetivo do modelo é minimizar a distância total acumulada por todas as equipes ao final da temporada, garantindo o atendimento estrito às restrições de mando de campo, como o limite de partidas consecutivas em casa ou como visitante. Apesar de, devido às restrições, ser uma adaptação mais complexa da TSP, a base teórica é feita em cima da modelagem clássica.

1.5 Aplicação na UFMG



Por fim, uma aplicação pensada para uso na UFMG (elaborada para o trabalho, não contém artigo científico modelando esse problema dessa forma) seria modelar o trabalho dos guardas de segurança em forma de uma TSP, a fim de minimizar as distâncias percorridas em uma ronda. Nesse problema, um prédio (ICEx, por exemplo) seria modelado como um grafo $G = (V, E)$, sendo as salas/pontos de vistoria do guarda os vértices V . O peso c_{ij} das arestas direcionadas E representaria a distância entre o ponto i e j . Nesse caso, usando o algoritmo, chegaríamos na rota mínima que garanta que o guarda passou por todos os pontos que precisava.



2 Interpretação do Modelo sem subciclos

Para solucionar o problema garantindo que não ocorram rotas isoladas e desconectadas da base, adota-se um modelo com restrições para subciclos. Sua diferença em relação ao modelo clássico é a adição da variável de fluxo f_{ij} , que indica o fluxo de mercadoria entre o nó i e j .

A lógica geral é que um fluxo de $n - 1$ é injetado na rede, e cada nó válido na rota consome 1 unidade desse fluxo, até que ele retorne ao nó de origem. As regras que validam essa modelagem são:

- $\sum_{(0,j) \in A} f_{0j} = (n - 1)$: A quantidade de fluxo que sai do nó de origem é $n - 1$.
- $\sum_{(i,j) \in A} f_{ij} - \sum_{(j,i) \in A} f_{ji} = 1, \quad \forall j \in N \setminus \{0\}$: Cada nó válido na rota consome 1 unidade de fluxo, enviando para o próximo nó exatamente 1 unidade a menos do que recebeu.
- $f_{ij} \leq (n - 1) \cdot x_{ij}$: Garante que o fluxo só existe quando $x_{ij} = 1$, ou seja, se existe rota naquela aresta.
- $f_{ij} \geq 0$: Garante que o fluxo é sempre positivo.

Com todas essas restrições, somadas às restrições do TSP clássico, temos um modelo que garante a inexistência de subciclos.

3 Instâncias Utilizadas

3.1 Instância 1: 21 Pontos (Original)

posx: [40, 25, 27, 3, 17, 33, 32, 26, 20, 31, 23, 38, 14, 45, 9, 19, 36, 7, 46, 30, 18]

posy: [5, 46, 39, 10, 20, 7, 5, 22, 31, 36, 45, 23, 28, 21, 14, 42, 29, 17, 4, 26, 30]

3.2 Instância 2: 30 Pontos

posx: [9, 25, 27, 3, 17, 33, 32, 26, 20, 31, 23, 38, 14, 45, 9, 19, 36, 7, 46, 30, 18, 40, 43, 42, 5, 10, 4, 47, 22, 11]

posy: [17, 31, 36, 7, 23, 28, 21, 40, 41, 14, 48, 49, 29, 34, 17, 4, 1, 6, 26, 46, 27, 32, 22, 44, 37, 20, 16, 11, 8, 39]

3.3 Instância 3: 50 Pontos

posx: [13, 25, 27, 3, 17, 33, 32, 26, 20, 31, 23, 38, 14, 45, 9, 19, 36, 7, 46, 30, 18, 40, 43, 42, 5, 10, 4, 47, 22, 11, 16, 41, 24, 12, 39, 6, 49, 21, 35, 34, 8, 48, 28, 1, 13, 37, 15, 29, 2, 44]

posy: [14, 40, 32, 22, 16, 21, 5, 13, 37, 15, 46, 10, 35, 29, 6, 36, 45, 33, 48, 4, 39, 18, 30, 23, 31, 42, 11, 25, 7, 20, 27, 19, 28, 41, 3, 34, 43, 47, 38, 17, 26, 44, 49, 2, 14, 1, 8, 12, 9, 24]

4 Discussão dos Resultados

4.1 Desempenho do Solver e Convergência

A Tabela 1 resume o desempenho do resolvidor CBC para as três instâncias testadas.

Instância	N (Nós)	Tempo de Resolução	Custo (FO)
Estudo de Caso	21	4.06 s	165.06
Gerada 1	30	12.79 s	235.35
Gerada 2	50	167538.04 s $\approx$ 46,5h	278.97



Table 1: Comparação de tempo e custo por tamanho de instância.

4.2 Análise da Complexidade e Esforço Computacional

A instância de 50 nós ofereceu uma demonstração prática da natureza *NP-hard* do TSP. O tempo de execução de aproximadamente 46,5 horas evidencia a explosão combinatória que ocorre ao expandir o espaço de busca.

De acordo com os logs de execução, o solver percorreu um total de 7.014.073 nós na árvore de busca e realizou mais de 832 milhões de iterações. Alguns pontos técnicos fundamentais para a convergência foram observados:

- **Strong Branching:** Técnica utilizada mais de 11 milhões de vezes para selecionar as ramificações mais promissoras da árvore, reduzindo o número total de nós explorados.
- **Custo Reduzido:** O algoritmo conseguiu fixar mais de 105 milhões de variáveis baseando-se no custo reduzido, o que simplificou o problema dinamicamente.
- **Prova de Otimalidade:** O longo tempo de execução não se deveu apenas à busca pela rota de custo 278.97, mas à necessidade matemática de provar que nenhuma das outras $(n - 1)!$ combinações era superior.

Este resultado valida a premissa de que, para problemas reais com $n > 50$, métodos exatos tornam-se proibitivos, sendo obrigatória a adoção de meta-heurísticas para obter soluções em tempos operacionais.

4.3 Visualização das Rotas

As figuras abaixo apresentam as rotas ótimas geradas pelo modelo. Nota-se que o algoritmo de eliminação de subciclos de Gavish e Graves garantiu a formação de um ciclo hamiltoniano único em todas as instâncias.

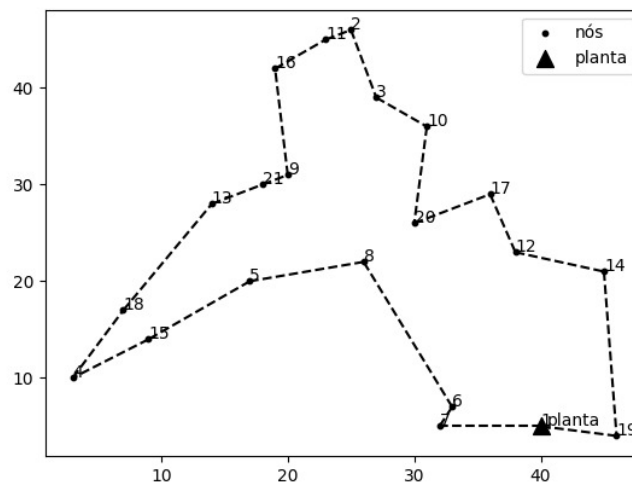


Figure 1: Rota final ótima para 21 pontos (Custo: 165.06).

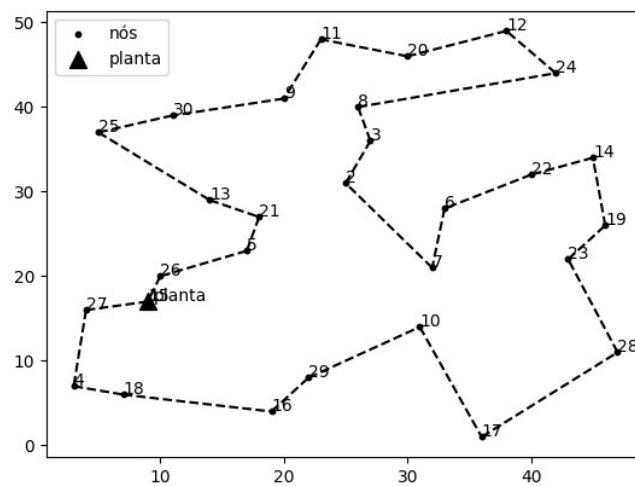


Figure 2: Rota final ótima para 30 pontos (Custo: 235.35).

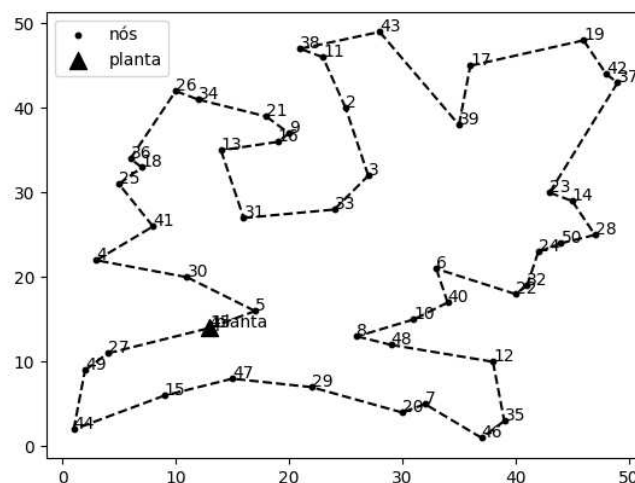


Figure 3: Rota final ótima para 50 pontos (Custo: 278.97) obtida após 46,5h de processamento.

References

- George Dantzig, Ray Fulkerson, and Selmer Johnson. Solution of a large-scale traveling-salesman problem. *Journal of the Operations Research Society of America*, 2(4):393–410, 1954.
- Kelly Easton, George Nemhauser, and Michael Trick. The traveling tournament problem: Description and benchmarks. *Principles and Practice of Constraint Programming—CP 2001*, pages 580–584, 2001.
- Richard M Karp. Mapping the genome: some combinatorial problems arising in molecular biology. *STOC '96: Proceedings of the twenty-eighth annual ACM symposium on Theory of computing*, pages 278–281, 1996.

Vangelis F Magirou. The efficient drilling of printed circuit boards. *Computers & Operations Research*, 13(6):715–723, 1986.